Lab I4 - Waypoints in Isaac Sim: red, green, blue

Aim. Write down a mission as three points and let a small ROS 2 node drive DiffBot to them in order, using only `/odom` for position. Three coloured squares painted on the floor of the Isaac Sim room mark the points.

Time. 30 min. **Needs.** Only this folder, an **NVIDIA RTX GPU** (tested: RTX 5050 Laptop, 8 GB, driver 592.82) and about 15 GB of disk for Isaac Sim, which `pixi run isaac-install` downloads once for all Isaac labs on the machine. By running the scripts you accept NVIDIA's Isaac Sim licence.

This folder is a complete project

Everything this lab needs is in this folder. Nothing from the other labs is required.

Requirements: Windows 10/11, PowerShell, pixi (no administrator rights needed; install steps are in README-SETUP.md), and an NVIDIA RTX GPU with a current driver; about 15 GB disk for Isaac Sim (shared by all Isaac labs, installed once into `C:\ix`); internet for the first install.

In this folder	
`isaac\`	the Isaac Sim scene scripts and ROS 2 nodes
`scripts\`	helper scripts and the automated tests
`pixi.toml`	the environment and every `pixi run ...` command; `README.md` this manual; `tools\replay.ps1` re-runs the manual's commands as a self-test

Set it up once. `pixi install` puts ROS 2 Jazzy into `C:\rlabs` (a few GB; pixi's package cache makes the second lab on the same machine fast). Open PowerShell in this folder and run:

```
pixi install
pixi run verify
```

Run one lab at a time, and everything from this folder.

Theory

Positions are in the `odom` frame; the robot starts at (0, 0) looking along +x:

Waypoint	Square	(x, y) in m
A	red	(1.5, 0.0)
B	green	(1.5, 1.5)
C	blue	(-0.5, 1.5)

The squares are flat boxes **without collision** - paint, not obstacles. The node `isaac\ros_nodes\waypoints.py` is a "go to a point" controller in 60 lines. For the current goal, every time `/odom` arrives:

```
heading_error = direction_to_goal - my_heading     (wrapped to -180 ... +180 degrees)
turn  w = 1.5 * heading_error   (limited to 1.0 rad/s)
drive v = only when |heading_error| < 0.5 rad, slower when close (at most 0.25 m/s)
closer than 0.12 m -> "arrived", next goal; after C: stop
```

`my_heading = 2 * atan2(z, w)` from the odometry quaternion. The coordinates are written twice (scene script and node): change both to move a square.

Procedure

1. T1 - Isaac Sim (window: room with the three squares).

```
pixi run isaac-p4 --gui
```

2. T2 - start the mission. The node prints `arrived at ...` for each point and `all waypoints visited: stopping`. (6-26 simulated seconds per leg; allow about a minute.)

```
pixi run isaac-waypoints
```

3. T3 - where did it stop? It should stand on the blue square, (-0.5, 1.5), within about 10 cm.

```
pixi run isaac-ros topic echo /odom --no-daemon --once --spin-time 5 --field pose.pose.position
```

```
# Ctrl+C in T1 and T2
```

Observations (recorded)

Quantity	Value
Arrival at A, B, C	about 6-9, 14-17 and 23-26 simulated seconds (three runs)
Final position	10 cm from the centre of C; moved 0.1 cm in the last 2 s (stopped)
Room	never left the 8 x 6 m room

Verification (own headless Isaac Sim, 3-5 min; close T1/T2 first)

```
pixi run test-isaac-p4
```

7 checks, judged from `/odom` only: the robot starts at the origin; it comes within 20 cm of A, B and C **in that order**; it stops after C (moves less than 3 cm in the last 2 s) standing on C; it never leaves the room.

Questions

1. Why does the node turn first and drive only when it faces the goal within 0.5 rad? What happens if it always drives?
2. What changes if `TOLERANCE` is 0.02 m? (Think of a proportional controller and wheel slip.)
3. A full navigator also plans around obstacles and reports failure. What does this node not have? When would that matter?
4. Add a fourth square D at (-0.5, 0.0) and a loop A -> B -> C -> D -> A. Which two files do you edit?
5. *(Extension)* Put a box between B and C and add a lidar to get round it.

If it does not work

Symptom	Fix
Node says "waiting for /odom"	T1 is not ready

Symptom	Fix
The mission does not repeat	the node stops after C; restart T2 (and T1, which resets the robot to the origin)